

B1 – DOCUMENTATION API

Campus Companion —Formation B3

Table des matières

1. Objet de l'annexe	1
2. Présentation du projet — Campus Companion Ynov.....	1
2.1 Contrainte à l'origine de l'architecture	2
2.2 Fonctionnalités principales (extrait README.md)	2
3. Architecture logicielle réutilisable.....	2
3.1 Stack technique (extrait README.md).....	2
3.2 Ports et exposition des services	2
3.3 Extrait — docker-compose.yml	3
3.4 Extrait — pom.xml (dépendances Maven).....	4
3.5 Consommation par trois clients indépendants	5
4. Briques logicielles et services métiers exposés par l'API.....	5
4.1 Exemple de brique — Auth (extrait API_DOCUMENTATION.md).....	6
4.2 Exemple de brique — Documents.....	6
4.3 Exemple de brique — Knowledge	6

1. Objet de l'annexe

Cette annexe rassemble les preuves techniques associées à la compétence C1.3 — « Programmation de briques et services logiciels. Conception de services métiers » — du référentiel RNCP 36463 (Concepteur Développeur d'Applications). Elle documente le service backend développé pour le projet Campus Companion Ynov, réalisé lors d'un Challenge de 48 heures en équipe à Ynov Campus (mars 2026).

Le référentiel exige de concevoir l'architecture d'une solution fiable en identifiant les spécificités d'une activité, afin de produire du logiciel générique réutilisable. Les pièces jointes ci-dessous en sont l'extrait : présentation du projet (README.md), configuration de la stack technique (docker-compose.yml, pom.xml) et documentation des briques logicielles exposées par l'API (API_DOCUMENTATION.md).

Les captures d'écran des trois applications clientes (mobile, web, desktop) font l'objet d'une annexe séparée.

2. Présentation du projet — Campus Companion Ynov

Campus Companion est un assistant conversationnel destiné aux étudiants du campus Ynov. Il répond aux questions administratives et pédagogiques courantes (absences, rattrapages, inscriptions, règlement intérieur...) à partir d'une base de connaissances alimentée par l'équipe pédagogique, et transmet automatiquement un ticket lorsque l'IA ne dispose pas d'une réponse fiable.

2.1 Contrainte à l'origine de l'architecture

La contrainte centrale du projet était la multiplicité des plateformes cibles : les étudiants utilisent leur mobile en déplacement, un navigateur web sur les postes du campus, et potentiellement un client desktop. Il fallait donc livrer trois clients distincts — mobile (Flutter/Dart), web (React) et desktop (Electron) — sans dupliquer la logique métier entre eux.

2.2 Fonctionnalités principales (extrait README.md)

- Chat IA via POST /api/chat, avec base de connaissances (RAG) et modèle Ollama (llama3.2) ;
- Fallback automatique : création d'un ticket pédagogique si l'IA ne trouve pas de réponse fiable ;
- Base documentaire avec upload et téléchargement de fichiers ;
- Base de connaissances avec ingestion de contenu et réindexation ;
- Authentification JWT avec rôles ADMIN / PEDAGOGUE ;
- Historique des interactions et recueil de feedback utilisateur (pouce haut / bas).

3. Architecture logicielle réutilisable

L'analyse métier a permis de séparer ce qui était spécifique à chaque plateforme (interfaces, interactions tactiles vs clavier, notifications push) de ce qui était commun aux trois clients : gestion des questions, des tickets, de la base documentaire, de l'historique et des utilisateurs. Cette séparation a guidé la décision de concentrer toute la logique métier dans un service backend générique, indépendant de tout client consommateur.

3.1 Stack technique (extrait README.md)

Composant	Technologie
Langage / Framework	Java 17, Spring Boot 4
Sécurité	Spring Security — JWT
Accès aux données	Spring Data JPA + Flyway (migrations)
Base de données	PostgreSQL
IA / RAG	Ollama — modèle par défaut llama3.2
Orchestration	Docker Compose (Postgres + Ollama + API)

3.2 Ports et exposition des services

Service	Port
API	8080
PostgreSQL	5435 (hôte) → 5432 (conteneur)
Ollama	11434

3.3 Extrait — docker-compose.yml

Cet extrait illustre la conception de l'environnement comme un ensemble de services indépendants et réutilisables (base de données, moteur IA, API), chacun remplaçable ou redéployable sans impacter les autres, et consommable de façon identique par les trois clients.

```
services:
  postgres:
    image: postgres:16-alpine
    container_name: ycompagnon-postgres
    restart: unless-stopped
    environment:
      POSTGRES_DB: db_ycompagnon
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: 123456
    ports:
      - "5435:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres -d db_ycompagnon"]
      interval: 5s
      timeout: 5s
      retries: 10

  ollama:
    image: ollama/ollama:latest
    container_name: ycompagnon-ollama
    restart: unless-stopped
    ports:
      - "11434:11434"
    volumes:
      - ollama_data:/root/.ollama

  api:
    image: maven:3.9.11-eclipse-temurin-17
    container_name: ycompagnon-api
    restart: unless-stopped
    working_dir: /app
    command: sh -c "chmod +x mvnw && ./mvnw spring-boot:run"
    depends_on:
      postgres:
        condition: service_healthy
      ollama:
        condition: service_started
    environment:
      SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/db_ycompagnon
      SPRING_DATASOURCE_USERNAME: postgres
      SPRING_DATASOURCE_PASSWORD: 123456
      OLLAMA_BASE_URL: http://ollama:11434
      OLLAMA_MODEL: llama3.2
      UPLOAD_DIR: /app/uploads
      SERVER_PORT: 8080
    ports:
      - "8080:8080"
    volumes:
      - ./:/app
      - maven_cache:/root/.m2
      - ./uploads:/app/uploads

volumes:
  postgres_data:
  ollama_data:
  maven_cache:
```

3.4 Extrait — pom.xml (dépendances Maven)

Le choix des dépendances traduit l'exigence de généricité : Spring Web/WebFlux pour exposer des briques REST standard, Spring Data JPA + Flyway pour un accès aux données versionné, Tika pour traiter tout format de fichier déposé sans logique spécifique par type, MapStruct/Lombok pour limiter le code répétitif des services métiers.

```
<dependencies>
  <!-- Sécurité -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
  </dependency>

  <!-- JWT -->
  <dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-api</artifactId>
    <version>0.12.6</version>
  </dependency>
  <dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-impl</artifactId>
    <version>0.12.6</version>
    <scope>runtime</scope>
  </dependency>
  <dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-jackson</artifactId>
    <version>0.12.6</version>
    <scope>runtime</scope>
  </dependency>

  <!-- Accès aux données -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-flyway</artifactId>
  </dependency>
  <dependency>
    <groupId>org.flywaydb</groupId>
    <artifactId>flyway-database-postgresql</artifactId>
    <version>11.7.2</version>
  </dependency>
  <dependency>
    <groupId>org.postgresql</groupId>
    <artifactId>postgresql</artifactId>
    <scope>runtime</scope>
  </dependency>

  <!-- Web -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-webflux</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-mail</artifactId>
  </dependency>
</dependencies>
```

```

<!-- Extraction de contenu indépendante du format -->
<dependency>
  <groupId>org.apache.tika</groupId>
  <artifactId>tika-core</artifactId>
  <version>3.1.0</version>
</dependency>
<dependency>
  <groupId>org.apache.tika</groupId>
  <artifactId>tika-parsers-standard-package</artifactId>
  <version>3.1.0</version>
</dependency>

<!-- Réduction du code répétitif des services -->
<dependency>
  <groupId>org.projectlombok</groupId>
  <artifactId>lombok</artifactId>
  <optional>true</optional>
</dependency>
<dependency>
  <groupId>org.mapstruct</groupId>
  <artifactId>mapstruct</artifactId>
  <version>1.6.3</version>
</dependency>
</dependencies>

```

3.5 Consommation par trois clients indépendants

Les applications Flutter (mobile/), React (web/) et Electron (desktop/) du dépôt ycompagnon-apps consomment la même base URL et les mêmes contrats JSON, sans hypothèse sur le client appelant. Aucune logique métier n'est dupliquée côté client : par exemple, la création automatique d'un ticket lorsque l'IA est incertaine est intégralement portée par l'API — le client reçoit simplement ticketCreated: true dans la réponse et adapte son affichage en conséquence.

4. Briques logicielles et services métiers exposés par l'API

La compétence visée porte sur la programmation de briques et services logiciels génériques. L'API Campus Companion est décomposée en 7 briques indépendantes, chacune correspondant à une activité métier précise et consommable telle quelle par les trois clients, sans adaptation ni logique dupliquée.

Brique / Module	Activité métier couverte	Endpoints
Auth	Inscription / connexion, émission d'un jeton JWT	2
Chat	Point d'entrée conversationnel de l'IA (RAG + fallback ticket)	1
Tickets	Cycle de vie des demandes pédagogiques (création, résolution, apprentissage IA)	7
Documents	Base documentaire (upload, téléchargement, métadonnées)	8
Knowledge	Base de connaissances IA (ingestion, liste, réindexation)	4
Interactions	Historique des échanges + feedback utilisateur	5
Users	Gestion des comptes et du push token mobile	6

Au total, 33 endpoints REST sont exposés, tous documentés selon le même format (corps de requête, réponse type, code d'accès) dans API_DOCUMENTATION.md — ce contrat homogène est ce qui permet à chaque brique d'être réutilisée sans connaissance de son implémentation interne.

4.1 Exemple de brique — Auth (extrait API_DOCUMENTATION.md)

La brique Auth encapsule toute la logique d'authentification (hachage BCrypt, émission JWT, rôles) derrière deux endpoints génériques, réutilisables par n'importe quel client sans connaître le détail de son fonctionnement interne.

```
POST /api/auth/register [Public]
{
  "email": "admin@ynov.com",
  "password": "password123",
  "role": "ADMIN"
}

POST /api/auth/login [Public]
→ 200 OK
{
  "token": "eyJhbGciOiJIUzI1NiJ9...",
  "type": "Bearer",
  "role": "ADMIN"
}
```

4.2 Exemple de brique — Documents

La brique Documents illustre la logique de "conception de services métiers" : upload, listing, téléchargement et suppression sont regroupés en une seule brique cohérente, avec indexation automatique pour le RAG déclenchée en interne — le client n'a besoin de connaître qu'un seul contrat d'appel, quel que soit le type de fichier déposé.

Endpoint	Accès	Description
GET /api/documents	ADMIN/PEDAGOGUE	Liste des documents, filtrable par type (PDF, TEXT)
POST /api/documents/upload	ADMIN/PEDAGOGUE	Upload multipart (50 Mo max) + indexation RAG automatique
GET /api/documents/{id}/download	Public	Téléchargement du fichier physique
DELETE /api/documents/{id}	ADMIN/PEDAGOGUE	Suppression du document et du fichier physique associé

4.3 Exemple de brique — Knowledge

La brique Knowledge est un service métier autonome : elle accepte du texte injecté directement ou du contenu extrait de fichiers (brique Documents), et le rend interrogeable par le moteur IA via un seul point d'entrée générique (POST /api/knowledge/ingest), réutilisable indépendamment de la source du contenu.

```
POST /api/knowledge/ingest
{
  "title": "Procédure rattrapage 2026",
  "type": "TEXT",
  "tags": "rattrapage,examen,session,note",
  "content": "Pour rattraper un examen, la moyenne doit etre",
  "sourceUrl": "kb://reglements/rattrapage"
}
```